3. Build

This implementation is to the specifications of the Pizzaria company as well as following our own internal design specifications. The design was built in Flowgorithm and saved as PizzaOrderHandler.fprg.

3.1 Instructions on running the program

These are step by step instructions for using the program

1. Open Flowgorithm on your PC and navigate at the top to: 'File' then click 'Open', then select the file titled 'PizzaOrderHandler.fprg'.

2. Press on the green arrow pointing right (play button) to run the program

3.2 Program flowcharts

Each figure shows one function of PizzaOrderHandler.fprg, shape by shape, including its comments

Key:

Comments – Dotted Line Boxes

Declare – Yellow box with black lines

Assign – Yellow Box

Output – Green Box

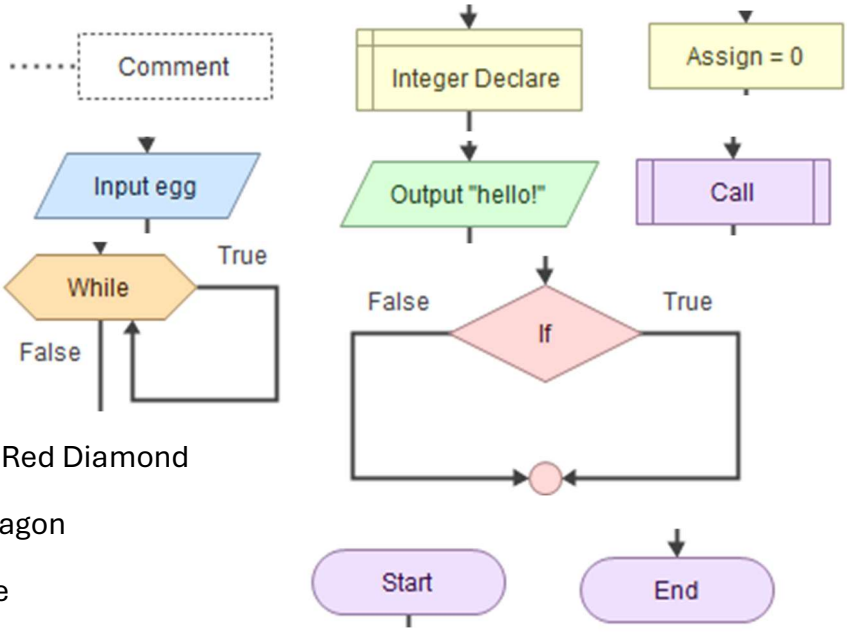
Input – Blue Box

Call(subroutine) – purple

'IF' Statement (true or false operator) - Red Diamond

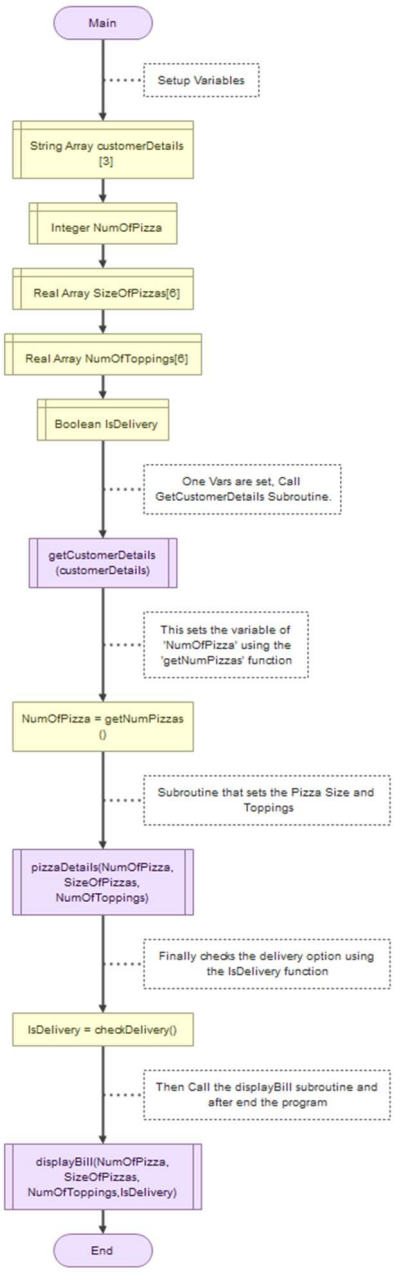
'While' 'Do' or 'For' loops – Orange Hexagon

Start/Return – purple rounded rectangle



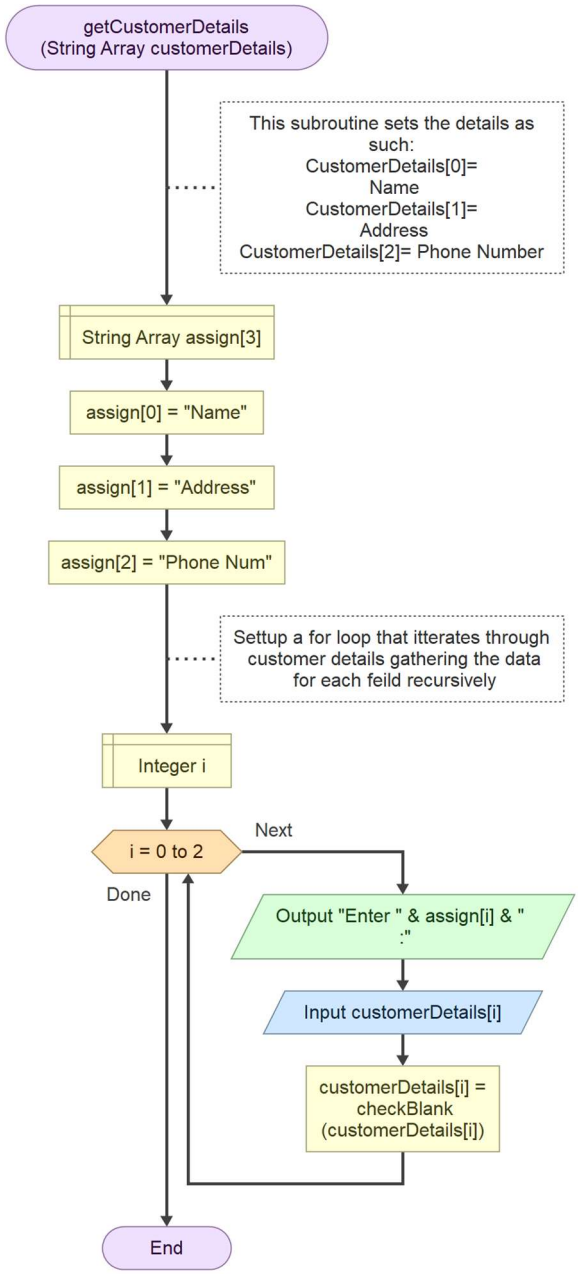
3.2.1 Main

Set up the variables and call each step in the ordering process sequentially, starting with getCustomerDetails:



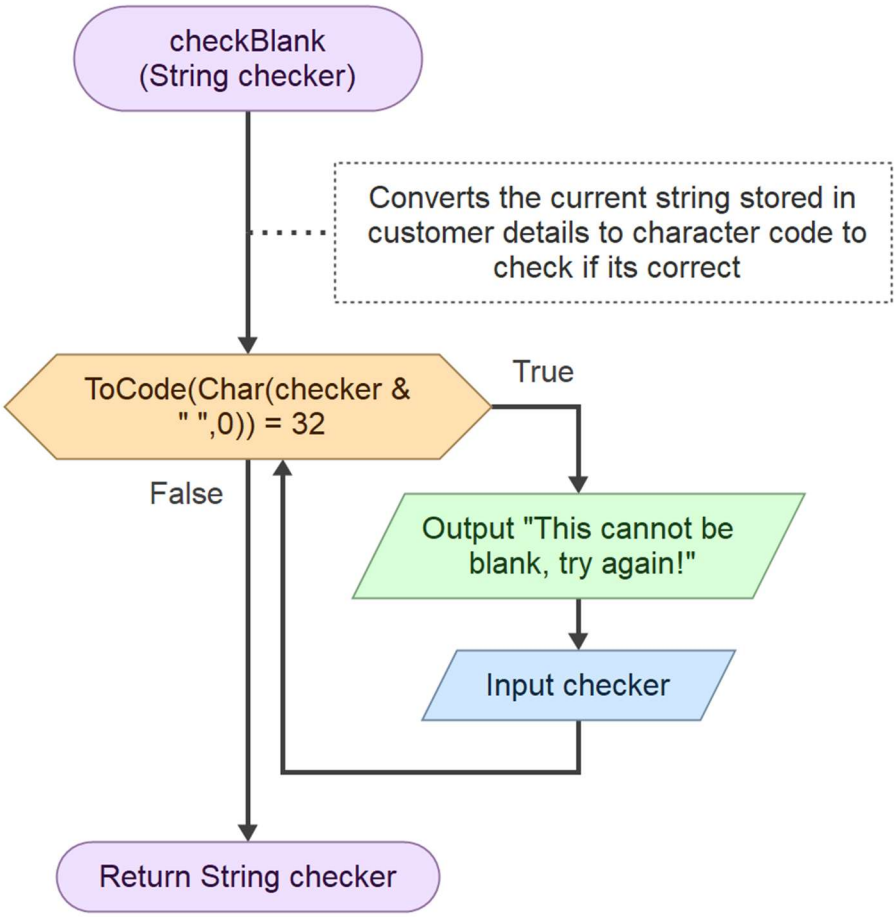
3.2.2 getCustomerDetails

This function Assigns the users inputs to the variable ‘customerDetails’ to be printed when the order is completed



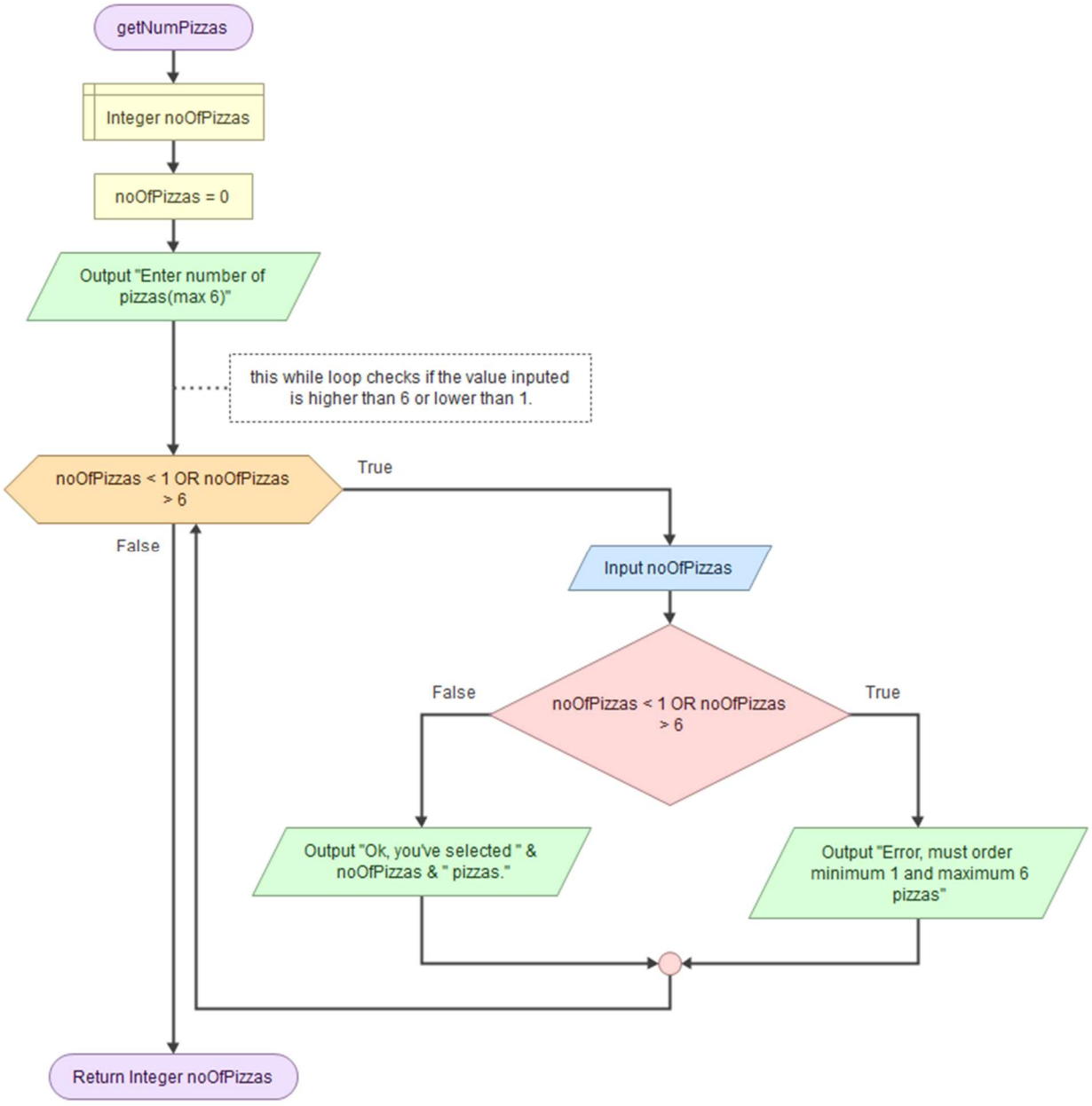
3.2.3 checkBlank

This is a simple function that uses an intrinsic function called ‘ToChar’ that converts the string to character code so that it can check to see if its value is blank, if so loop and tell the user to try again.



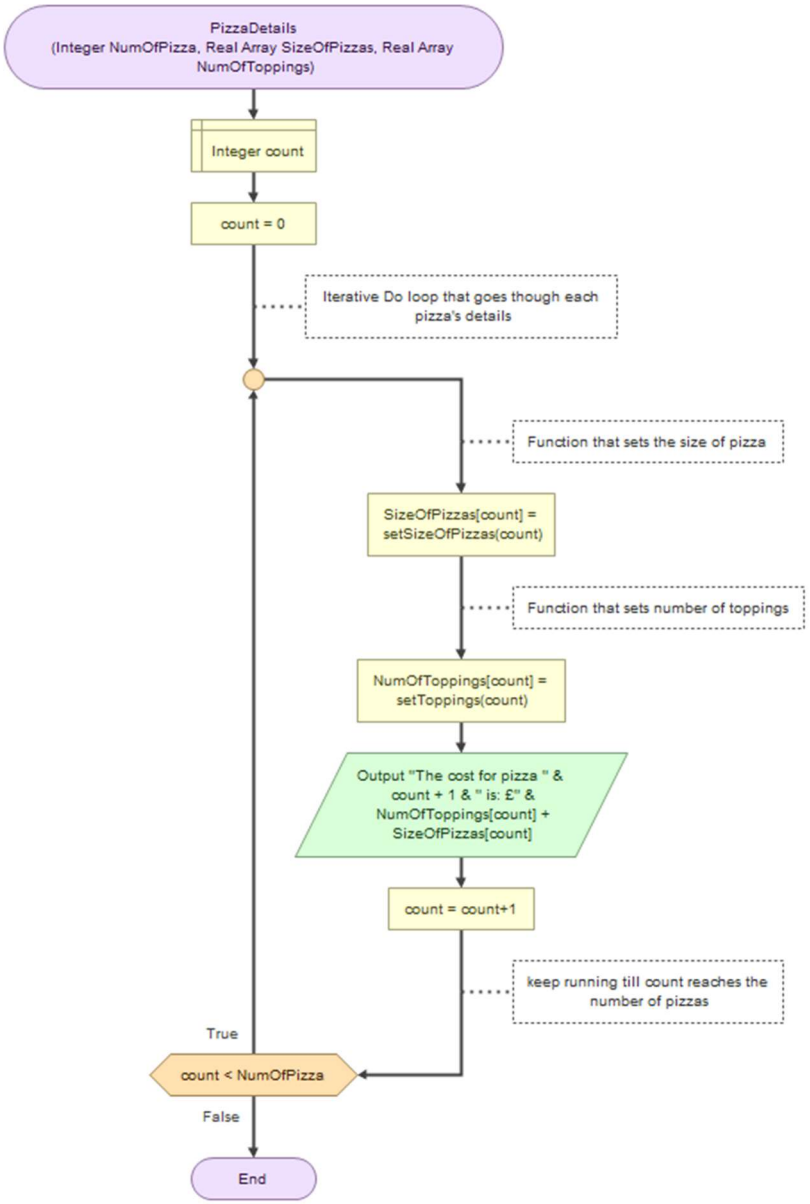
3.2.4 getNumPizzas

This function assigns the variable numPizza the amount of pizzas the customer has ordered up to 6. It checks if the value inputted is higher than 6 or lower than 0 before continuing.



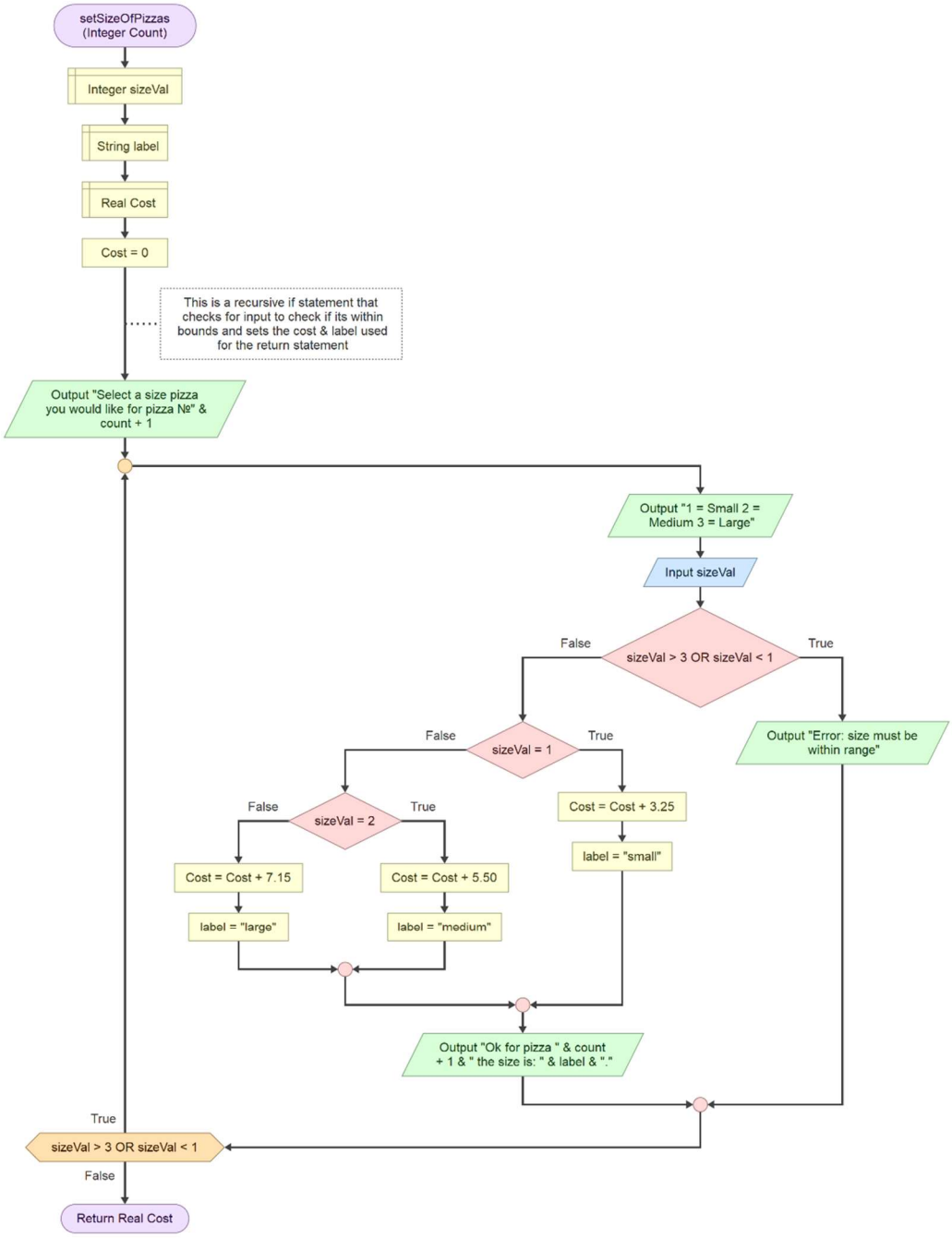
3.2.5 pizzaDetails

This subroutine purpose is to call two other functions (SizeOfPizzas & NumOfToppings) which set the size and amount of extra toppings if specified. It does this by incrementing a counter which runs in a loop until the 'DO' loop finds the counter is the same as the Number of pizzas ordered.



3.2.6 setSizeOfPizzas

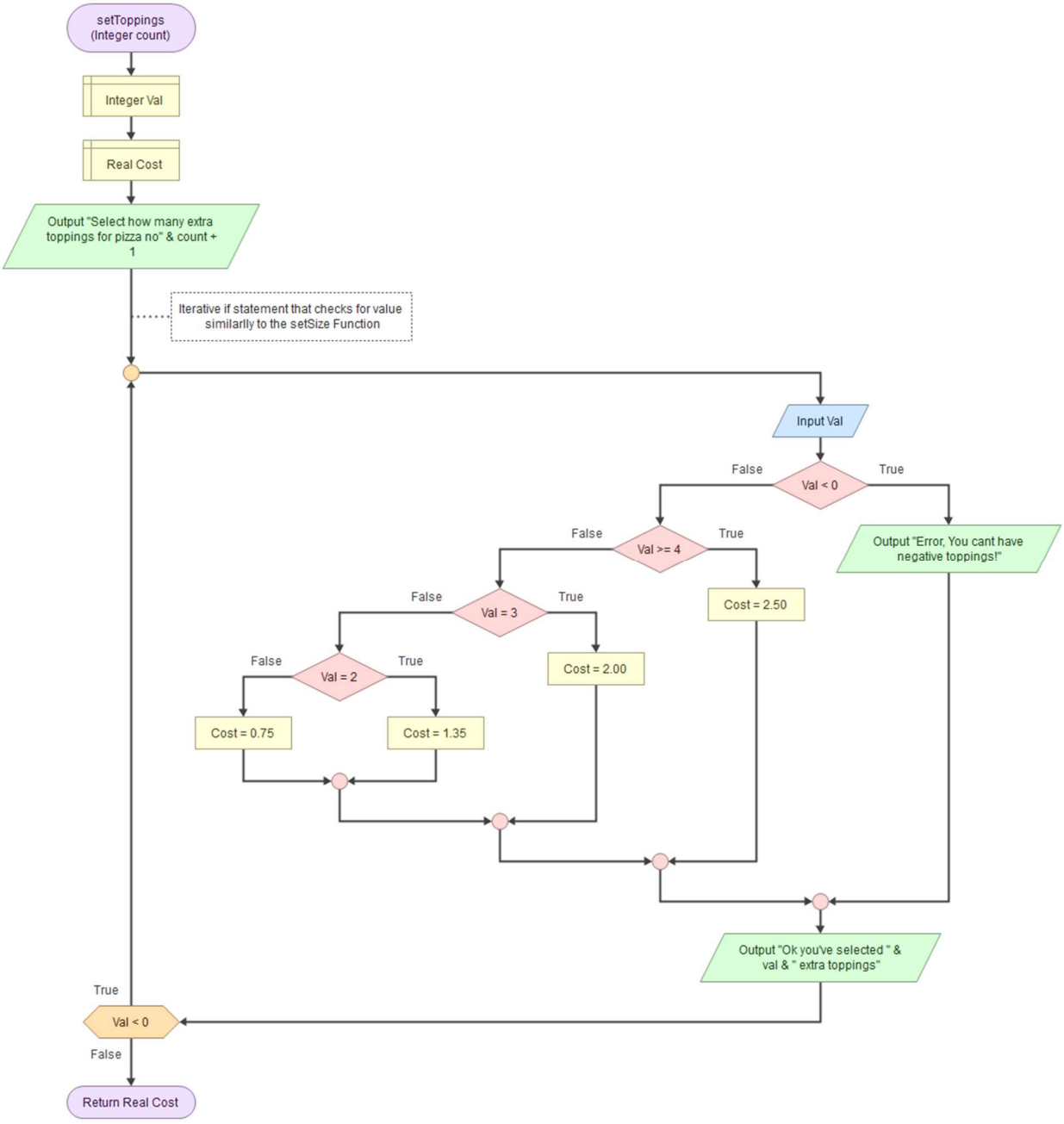
This is a function that uses a nested IF statement that validated the inputted data and sets the variable array SizeOfPizzas to the cost of the size selected. It inputs the cost into the correct pizzas section by using the counter from ‘PizzaDetails’ as a reference point to the index in the array. This means it’s reused for each pizza ordered.



3.2.7 setToppings

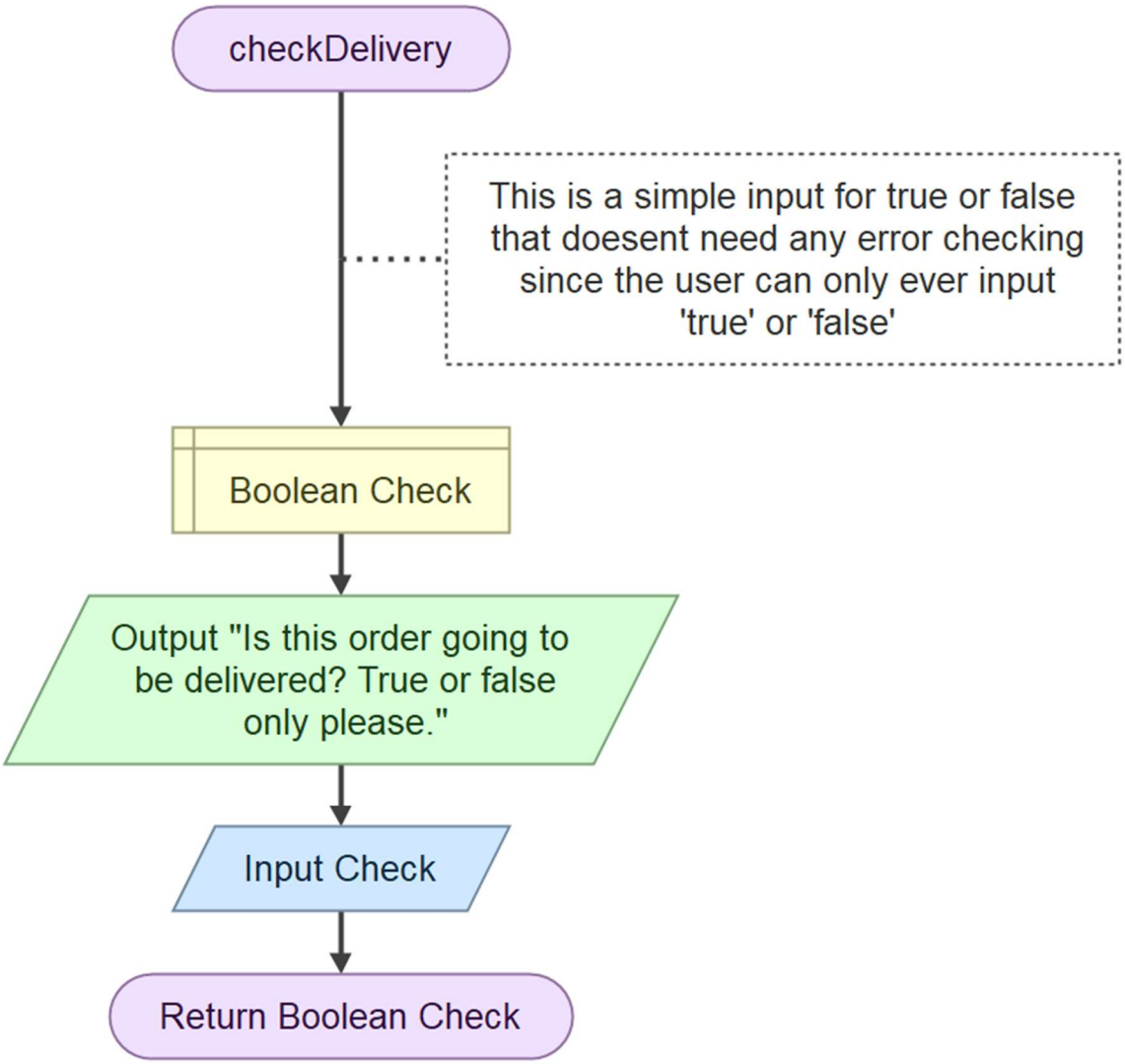
This function’s purpose is much the same as SetSizeOfPizzas just with the extra toppings instead, it’s functionally the same but just with one more if statement and no

upper limit to the amount of toppings, just the cost gets limited to £2.50.



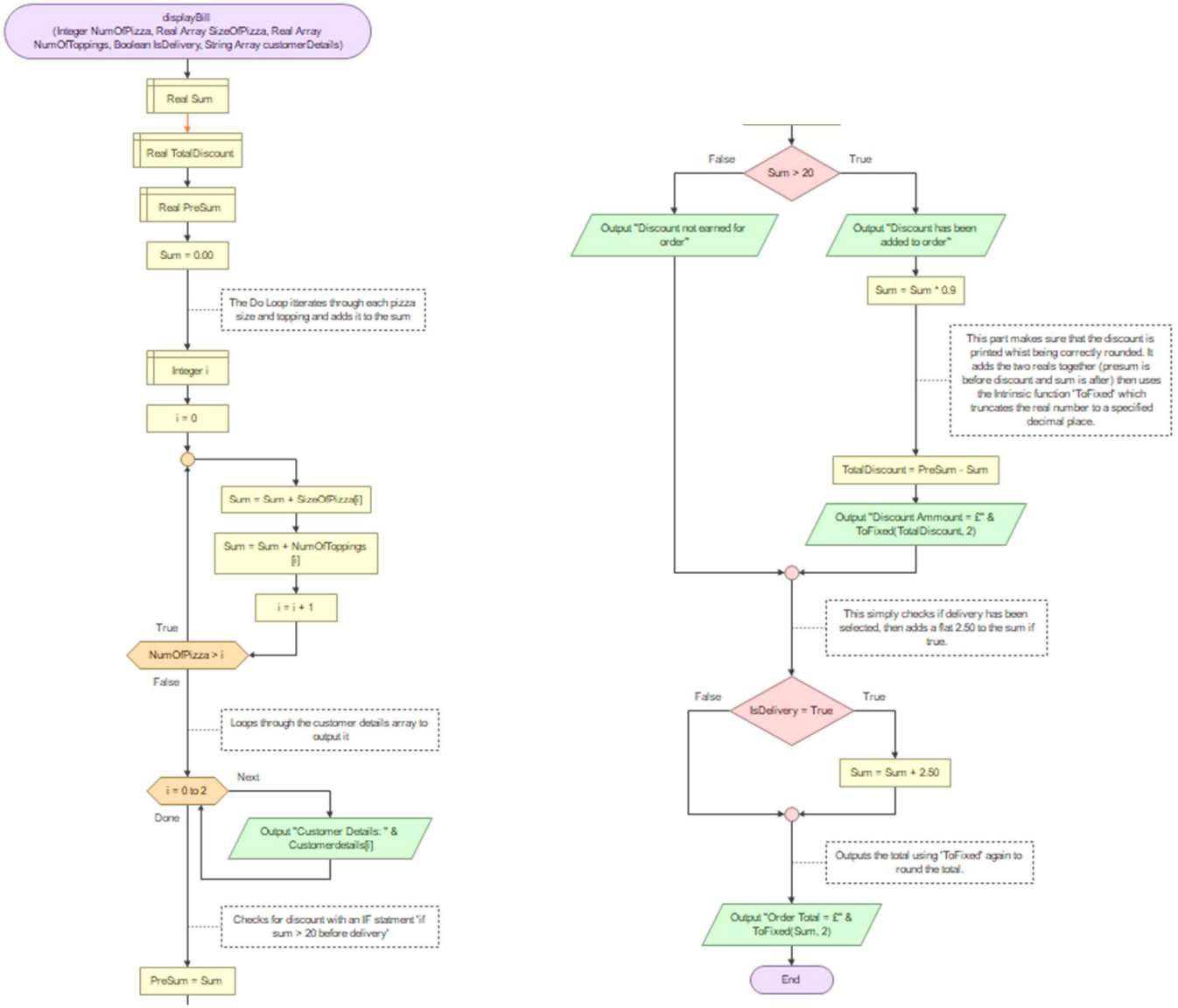
3.2.8 checkDelivery

The penultimate and the simplest function of the whole program. All it does is ask for an input that is returned into a Boolean, no error checking necessary since inputs can't place exceptionally wrong data (like putting a string into an integer) so it's really quite simple. The only reason why this check is separate to the final 'displayBill' subroutine is that debugging is easier as well as making the code more human readable.



3.2.9 displayBill

This is the final subroutine and ultimate part of the program that recursively adds all the cost information from both arrays (SizeOfPizzas & NumOfToppings), Outputs the customer details, then checks if a discount is earned and calculates it and displays the amount saved if true, before finally checking the delivery and if it's true then adding a flat 2.50.



3.3 How the build matches with the design

Design	Implementation
Main() Flowchart design Subroutines	Main implementation is the same as the mockup Draw.io design 'to a tee'. Every function and subroutine is called at the same time and named the same as is specified in the mockup.
Arrays	customerDetails, SizeOfPizzas and NumOfToppings are all declared in main and follow the same naming convention as is specified in section 2.4
Data Validation Data accuracy	The function 'checkBlank' is exclusively made to ensure the data inputted into 'customerDetails' is not blank The Total cost and Savings amount is displayed using the ToFixed() intrinsic function that corrects the savings from being three decimal places (which doesn't make sense for currency) to two decimal places.

3.4 Programming conventions followed

- Comments – Labels that tell the other programmers and outside parties how the program works for less obfuscated code
- Accurate names – the variables subroutines and functions are all labeled appropriately and exactly as to ensure less confusion when reading or changing code e.g.: 'customerDetails' 'DisplayBill' or 'CheckDelivery'.
- Subroutine and function efficiency - One job for subroutine and usually multiple similar jobs for each function, so the code is cleaner.
- Syntax – No basic syntax or lexical errors; everything is defined correctly before usage.
- Variables – variables that aren't needed in multiple subroutines/functions are defined locally instead of globally.

Small improvements to be made: Could check the input for customerDetails to ensure the exact correct data is being inputted (i.e. phone number char length or no numbers in name)

3.5 Sample Run

This is the input and output from a typical order, Lines that are highlighted in yellow are inputs from staff members:

```
Enter Name  :
Genry
Enter Address  :
123 fake street
Enter Phone Num  :
0712345678
Enter number of pizzas(max 6)
2
Ok, you've selected 2 pizzas.
Select a size pizza you would like for pizza №1
1 = Small 2 = Medium 3 = Large
2
Ok for pizza 1 the size is: medium.
Select how many extra toppings for pizza no1
5
Ok you've selected 5 extra toppings
The cost for pizza 1 is: £8
Select a size pizza you would like for pizza №2
1 = Small 2 = Medium 3 = Large
3
Ok for pizza 2 the size is: large.
Select how many extra toppings for pizza no2
2
Ok you've selected 2 extra toppings
The cost for pizza 2 is: £8.5
Is this order going to be delivered? True or false only please.
true
Customer Details: Genry
Customer Details: 123 fake street
Customer Details: 0712345678
Discount not earned for order
Order Total = £19.00
```